

Technical Fluency for Startups

17 sessions · 60–90 minutes each · one project throughout

What this course produces

Two outcomes, weighted equally:

1. You can speak intelligently about systems and products. You sit in a room with engineers and follow the conversation without translating. You read an architecture diagram. You know what someone means by "we'd need to refactor the auth flow," "that's a queue problem," or "the eval scores dropped." The vocabulary is yours, not something you nod through.

2. You can build rudimentary products and be useful at an early-stage company. You pull your own data instead of filing a ticket. You ship a working prototype in a weekend. You open a codebase and find where things live. You debug well enough to know which layer failed before you ask anyone for help.

What this course is not: training to be a software engineer. No algorithms, no data structures, no technical interview prep. That omission is deliberate and it's what makes 17 sessions enough.

The project: Mini AI CRM

One application, extended across the whole course, rather than seventeen disconnected tutorials. The point isn't CRM software — it's that building this one thing forces you through nearly every concept worth having.

By the end it will let someone:

1. Create an account and log in
2. Add companies and notes about them
3. Store everything in a real database
4. Ask an AI model to analyze the notes
5. Get back structured output that's saved and viewable
6. Do all of it at a public URL

Target architecture:



Every box in that diagram should feel concrete by the end.

Daily method

- **20–30 min — Read.** The assigned material, nothing more. Resist the urge to follow every link.
- **30–45 min — Touch it.** Run something, query something, break something. Reading alone doesn't stick.
- **10 min — Explain.** Close the docs. Say out loud what you just learned. Draw the architecture from memory.
- **5 min — Glossary.** Add the session's terms to one running document, **in your own words**. Never copy a definition.

That last step is the one people skip, and it's the one that converts "I read about webhooks" into "I can use the word webhook in a meeting and mean it."

PART I — THE MACHINE

Sessions 1–4. How software works, before you build any.

Session 1 — How the internet actually works

~75 min

Why this matters

Every other session assumes this one. "The frontend calls the backend" is a sentence you'll hear a hundred times, and it's meaningless until you've traced what physically happens between a keystroke and a rendered page. This is also the session that makes deployment, latency, and API keys make sense later — all three are consequences of the request/response model.

Objectives

By the end you should be able to:

- Narrate what happens between typing a URL and seeing a page, out loud, without notes
- Explain the difference between your machine and a server, and why `localhost` is invisible to everyone else
- Look at a status code and know roughly what went wrong
- Say what "the frontend made a request to the backend" means in mechanical terms

Read

- [MDN — Overview of HTTP](#) — from the top through the basic HTTP flow (25 min)
- [MDN — HTTP Messages](#) — only the request/response structure (10 min)
- [Cloudflare — What is DNS?](#) (10 min)

Deliberately skip: HTTP/2, HTTP/3, proxies, TCP internals, TLS handshakes. All real, all irrelevant to you now.

Do

Ask Claude: *"Walk me through exactly what happens when I type doordash.com into Chrome and load the homepage. Cover DNS, browser, server, frontend, backend, database, APIs, and HTTP at the level of a startup founder."*

Then close it and draw the whole chain on paper from memory. Then explain it out loud to nobody. The drawing and the explaining are the session — the reading is just input.

Second thing: open any website, open Chrome DevTools (`Cmd+Option+I`), click the **Network** tab, and reload. You're now watching dozens of real HTTP requests. Click one. Look at the method, the status, the headers. You don't need to understand it yet — you need to know this panel exists, because Session 15 lives here.

Go deeper

- [What happens when you type a URL](#) — famously exhaustive. Skim it; don't try to finish it.

- [High Performance Browser Networking](#) — free book. Chapter 1 only, if you want the layer below HTTP.

Glossary

Term	What it means
Client	Whatever is making the request. Usually a browser.
Server	A computer that receives requests and returns responses.
Frontend	Code running in the user's browser.
Backend	Code running on a server the user can't see or touch.
HTTP	The protocol — the agreed rules for how clients and servers talk.
Request / Response	The two halves of every web interaction.
DNS	The system that turns a domain name into a machine address.
Domain	The human-readable name (guhan.space).
IP address	The numeric address of a machine on a network.
Localhost	Your own computer, reachable only by you.
Port	A numbered channel (3000, 443) so multiple servers can run on one machine.
Status code	A number summarizing the outcome. 200 OK, 404 not found, 500 server error.
Header	Metadata attached to a request or response.
Payload / Body	The actual content being sent.
Cloud	Someone else's computers, rented by the hour.

Done when

You can say *"the frontend made an HTTP request to the backend, which queried the database and returned a response"* and every noun in that sentence points at something specific.

Session 2 — APIs and JSON

~90 min

Why this matters

This is one of the two highest-value sessions in the course. Most startup products are, structurally, interfaces that retrieve, transform, store, and send data — and APIs are how the retrieving and sending happen. Once APIs and JSON are transparent to you, a large share of "how does that product work?" questions answer themselves.

It's also the most immediately useful thing in a business role. "Can we pull that from their API?" is a question you'll need to answer, and answering it means reading someone's documentation and forming a view.

Objectives

By the end you should be able to:

- Read an API's documentation and work out whether it can do what you need
- Look at a JSON blob and describe its structure out loud
- Make a real API call yourself and read what came back
- Explain what an API key is and why it's a secret
- Explain the difference between an API call and a webhook

Read

- [MDN — Introduction to Web APIs](#) (20 min)
- [MDN — Working with JSON](#) (15 min)
- [Cloudflare — What is a REST API?](#) (10 min)

Do

Ask Claude Code:

Create a small script that calls a free public API, prints the status code, and prints the returned JSON. Under 30 lines. Explain every line before I run it.

Run it. Then break it on purpose, one thing at a time: wrong endpoint, missing parameter, malformed URL. Note what each failure looks like — you're building a library of failure shapes that'll pay off in Session 15.

Then ask: *"Show me exactly which part of this is the HTTP request and which part is the response."*

The picture to internalize

```
None
Your program
  ↓
GET https://api.example.com/company/123
  ↓
API server
  ↓
{ "id": 123, "name": "Acme" }
```

An API is not magic. It is software sending structured requests to other software and getting structured answers back.

Go deeper

- [Stripe API Reference](#) — the best-written API documentation in the industry. Read a few pages purely to calibrate what good looks like. Every time you evaluate a vendor's API later, this is your benchmark.
- [Public APIs list](#) — a directory of free APIs. Useful when you want something real to call.
- [Postman](#) — a GUI for making API requests without writing code. Worth twenty minutes; it makes the request/response cycle tangible.

Glossary

Term	What it means
API	A defined way for one piece of software to ask another for something.
REST API	The dominant convention: structure your API around URLs and HTTP methods.
Endpoint	One specific URL an API exposes.
GET / POST / PATCH / DELETE	Read / create / update / remove.

Term	What it means
Header	Metadata on a request — auth tokens, content type.
Body	The payload.
API key	A secret string proving you're allowed to call this API.
JSON	The standard text format for structured data: keys, values, arrays, nesting.
SDK	A library wrapping an API so you write less boilerplate.
Webhook	The reverse of an API call — the service calls <i>you</i> when something happens.
Rate limit	A cap on requests per window.
Pagination	Returning results in pages because the full set is too large.
Idempotent	Safe to repeat. GET is; POST usually isn't.

Done when

Someone says *"hit the Stripe API with a POST and parse the returned JSON"* and every word of it is transparent.

Session 3 — Databases and SQL, part 1

~90 min

Why this matters

SQL is the single highest-ROI concrete skill on this list for someone in a business role at a startup. Not because engineers can't write it — because you'll want an answer *during a meeting*, and asking someone else means a two-day round trip. The person who can answer "how many March signups are still active?" in thirty seconds becomes indispensable fast.

It's also fifty years old, universal, and not going anywhere. Nothing else you learn here has that shelf life.

You have an advantage most beginners don't: you already think in tables, cohorts, and joins from financial analysis. You're learning notation for concepts you have.

Objectives

By the end you should be able to:

- Look at a set of tables and explain how they relate
- Write a SELECT with filtering and sorting from a blank page, no reference
- Explain what a primary key and a foreign key are, and why they exist
- Describe what CRUD means in terms of what a user does in an app

The abstraction, first

A database is a set of tables. Tables have rows and columns. Tables point at each other using IDs.

```
None
USERS
id | name | email
-----
1  | Bob  | bob@x.com
2  | Sue  | sue@x.com

COMPANIES
id | user_id | name
-----
1  | 2       | Stripe
2  | 2       | OpenAI
3  | 1       | Airbnb
```

`user_id` is what connects them. That one idea — a column in one table pointing at a row in another — is most of what "relational database" means.

Read and do

- [SQLBolt](#) — lessons 1 through 6. Interactive, instant feedback, no setup. Do these first.
- [PostgreSQL — The SQL Language](#) — sections 2.2 through 2.5: concepts, creating a table, populating it, querying it.

Go deeper

- [Mode SQL Tutorial — Basic](#) — same ground, different explanation. Useful if SQLBolt's framing doesn't click.
- [Select Star SQL](#) — an interactive book that teaches SQL through a real dataset. Genuinely well-written.

Glossary

Term	What it means
Database	Organized, persistent storage.
Relational database	One organized as tables that reference each other.
Postgres	The most common open-source relational database. The default choice.
Table / row / column	Spreadsheet / record / field.
Schema	The structure — which tables exist and what columns they have.
Primary key	The unique ID of a row.
Foreign key	A column pointing at another table's primary key.
Query	A question you ask the database.
CRUD	Create, read, update, delete. The four operations.
SELECT	Choose which columns you want.
WHERE	Filter which rows.
ORDER BY / LIMIT	Sort / cap the number returned.
DISTINCT	Remove duplicate results.

Done when

You can write a SELECT with a WHERE clause from a blank page, no reference, no hesitation.

Session 4 — Databases and SQL, part 2

~90 min

Why this matters

Session 3 got you querying one table. This one gets you answering business questions, which almost always requires combining tables and summarizing. Joins and aggregates are the actual skill; everything before was setup.

This is also the session with a hard gate. Don't move past it until you clear the checkpoint — every later session assumes you can reason about data relationships.

Objectives

By the end you should be able to:

- Write a two-table join with a GROUP BY, from a blank page, unassisted
- Explain the difference between an inner join and a left join, and when the difference bites
- Design a simple schema for a product you're describing — decide what tables exist and how they connect
- Explain, roughly, why one query is slow and another is fast

Read and do

- [SQLBolt](#) — lessons 6 through 13: joins, NULLs, aggregates, order of execution.
- [PostgreSQL tutorial](#) — sections 2.6 (joins) and 2.7 (aggregates).
- [Mode SQL Tutorial — Intermediate](#) — the clearest free explanation of GROUP BY and joins available anywhere.

The two queries to write cold

```
SQL
SELECT *
FROM companies
WHERE industry = 'AI';
```

```
SQL
SELECT users.name, COUNT(companies.id)
FROM users
JOIN companies ON users.id = companies.user_id
```

```
GROUP BY users.name;
```

Exercise

Design the CRM schema yourself, on paper, before consulting anything. Three tables: users, companies, notes. Write down every column and every foreign key.

Then answer: *how would you retrieve every note belonging to Stripe?* Write the SQL first. Then check it.

Go deeper

- [Use The Index, Luke](#) — why some queries are slow and what an index actually does. First chapter only; save the rest until you have a slow query of your own.
- [PostgreSQL — Explain](#) — how to ask the database why it did what it did. Skim so you know it exists.

Glossary

Term	What it means
JOIN	Combine rows from two tables using a shared key.
INNER JOIN	Only rows with a match in both tables.
LEFT JOIN	Every row from the left table, matched or not.
GROUP BY	Collapse rows into groups so you can aggregate them.
Aggregate	COUNT, SUM, AVG, MIN, MAX.
HAVING	Like WHERE, but filters groups after aggregation.
NULL	Missing or unknown. Not zero, not empty string. Behaves surprisingly — this is the classic beginner trap.
Index	A lookup structure that makes certain queries dramatically faster.

Term	What it means
Migration	A versioned change to the database structure.
INSERT / UPDATE / DELETE	Add / change / remove rows.
Subquery / CTE	A query inside a query. CTEs (<code>WITH ... AS</code>) are the readable version.
Normalization	Structuring data so nothing is stored twice.

Done when

You write a two-table join with a GROUP BY, from blank, unassisted. Hard gate. Don't skip it.

PART II — BUILDING IT

Sessions 5–10. Frontend, backend, data, auth, deployment.

Session 5 — Terminal, Git, and GitHub

~60 min

Why this matters

Git is how every software team on earth collaborates, and the vocabulary around it — branch, PR, merge conflict, code review — makes up a large fraction of everyday engineering conversation. You don't need mastery. You need the workflow to stop being mysterious, and you need to have felt a merge conflict once so the term isn't abstract.

Objectives

By the end you should be able to:

- Create a branch, commit, push, open a pull request, and merge it
- Deliberately create a merge conflict and resolve it
- Explain what a PR is *for* — not just mechanically, but why teams work this way
- Read a repo's commit history and get a sense of what's been happening
- Know why `node_modules` and `.env` are in `.gitignore`

Read and do

- [GitHub — Hello World](#) — actually do the tutorial.
- [GitHub Flow](#) — read once. This is the collaboration model.

Commands worth having

```
Shell
pwd  ls  cd  mkdir          # moving around
git status  git diff        # what has changed
git add  git commit  git push # save and share
git pull                                # get others' changes
git branch  git checkout    # parallel work
git log                                # history
```

Exercise

In a scratch repo: branch off main, change a file, commit, push, open a PR against main, merge it.

Then create a merge conflict on purpose — edit the same line on two branches and merge them. Resolve it. The conflict is the point. It looks alarming until you've done it once, at which point it's a five-minute annoyance forever after.

Go deeper

- [Oh Shit, Git!?!](#) — plain-English fixes for the specific ways git goes wrong. Bookmark permanently.
- [Learn Git Branching](#) — visual and interactive. Do it if branching stays fuzzy.
- [Pro Git](#) — the free official book. Chapters 1–3 only, and only if you want the model underneath the commands.

Glossary

Term	What it means
Repository	A project tracked by git.
Commit	A saved snapshot, with a message.
Branch	A parallel line of work.
Main	The primary branch, the one that gets deployed.

Term	What it means
Pull request (PR)	A proposal to merge a branch, plus the discussion around it.
Merge	Combining branches together.
Merge conflict	Two branches changed the same lines; a human has to decide.
Remote / origin	The server copy; origin is its default name.
Clone / fork	Copy a repo locally / copy it under your own account.
.gitignore	Files git should never track — secrets, dependencies, build output.
Code review	Someone reading your PR before it merges.
Revert	Undo a commit, safely, with a record.

Done when

"Clone the repo, branch off main, commit your changes, push, and open a PR" is a sentence you could act on immediately.

Session 6 — Frontend, React, and Next.js

~90 min

Why this matters

You'll spend more time looking at frontend code than any other kind, because it's where the product visibly lives. You do not need to be good at React. You need to open a component file and roughly know what it does, and understand why the industry organizes interfaces this way.

The core idea worth taking away: React inverted how interfaces get built. Before it, you found elements on the page and changed them one at a time, which became unmanageable at scale. React lets you describe what the screen should look like given some data, and handles the updating. Every modern frontend framework works on that principle now.

Objectives

By the end you should be able to:

- Open a React component and describe what it renders
- Explain what state is and why a page "remembers" something
- Predict which URL a given file will produce in Next.js
- Explain the difference between a library and a framework
- Point at a file and say "this is what produces that page"

The framing

```
None
HTML      = structure
CSS        = appearance
JavaScript = behavior
React      = building interfaces out of reusable components
Next.js    = a framework around React adding routing, server code, and builds
```

Do **not** try to learn React properly here. Recognition, not mastery.

Read

- [Next.js — Installation](#) — follow it, create the CRM app
- [Next.js — Layouts and Pages](#) — read and experiment, 20 min
- [Next.js — Project Structure](#) — orient yourself, don't memorize

Build

Three pages, zero styling effort:

```
None
/                dashboard
/companies       list of companies
/companies/new   add-company form
```

Tell Claude Code: *"Keep this extremely simple. I care about architecture, not visual polish."*

Then inspect

Spend a real 15 minutes asking:

Explain this application file by file. Start with what matters most and skip config unless it's important.

When I load /companies, exactly which code produces what I see?

This inspection step is more valuable than the building step. The building is fast; understanding what got built is the skill.

Go deeper

- [React — Quick Start](#) — the official tutorial. Work through it if components and state stay fuzzy after this session.
- [React — Thinking in React](#) — short, and the single best explanation of *why* components are the unit.
- [Tailwind CSS docs](#) — pure reference. Look up class names when they confuse you; don't read it through.

Glossary

Term	What it means
Component	A reusable piece of interface, written as a function.
Props	Data passed into a component from outside.
State	Data a component holds and can change.
JSX	HTML-like syntax written inside JavaScript.
Hook	A React function like <code>useState</code> or <code>useEffect</code> that adds capability.
Routing	Mapping URLs to pages. In Next.js, folders <i>are</i> routes.
Server component / client component	Runs on the server before HTML is sent / runs in the browser.
Hydration	The browser taking over server-rendered HTML and making it interactive.
Rendering	Turning code into what's displayed.

Term	What it means
Build	Converting source files into what actually gets served.
Bundle	The packaged JavaScript sent to the browser. Size affects load time.
Framework vs library	A framework calls your code; you call a library.
Responsive	Adapting layout to screen size.

Done when

You can point at a file and explain what page it produces and roughly how it reaches the browser.

Session 7 — Backend and API routes

~90 min

Why this matters

This is where Session 1 stops being theory. You'll build an endpoint, call it from a form, and then *watch the request happen* in DevTools. That moment — seeing your own data leave the browser and come back — is the one that makes the whole architecture concrete.

It's also where the frontend/backend split stops being an arbitrary distinction and becomes obviously necessary: some things simply cannot happen in a browser, because the browser belongs to the user.

Objectives

By the end you should be able to:

- Explain what a backend does that a frontend cannot, and why
- Build a simple endpoint that receives data and returns a response
- Open DevTools, find a specific request, and read its method, status, payload, and response
- Explain what a 400 means versus a 500, and who's at fault in each case

Read

- [Next.js — Route Handlers](#)
- [MDN — HTTP response status codes](#) — skim the categories, memorize nothing

Build

One endpoint, no database yet:

```
None
POST /api/companies

in: { "name": "Stripe" }
out: { "success": true, "company": "Stripe" }
```

Wire your add-company form to it.

The exercise that matters most here

Open Chrome DevTools → **Network** tab. Submit the form. Click your request. Find:

- the method
- the URL
- the status code
- the request payload
- the response body

Reference: [Chrome DevTools — Inspect Network Activity](#)

You are now watching **frontend** → **HTTP** → **backend** → **HTTP** → **frontend**. Spend twenty minutes here. Click through requests. Change the form input and watch the payload change. This panel is where you'll spend most of Session 15, and where most real debugging starts.

Go deeper

- [MDN — Fetch API](#) — how the frontend makes requests. Worth understanding since you'll see **fetch** constantly.
- [Chrome DevTools — Overview](#) — tour the other panels so you know what's available.

Glossary

Term	What it means
Route handler / API route	Server-side code responding to requests at a URL.
Request payload	What the client sent.
Response body	What the server sent back.
Serialization	Converting data into a transmittable format, usually JSON.
CORS	Browser rules about which sites may call which APIs. You will meet this as an error message.
Middleware	Code that runs on every request before your handler does.
Validation	Checking incoming data is what you expected. Never trust the client.
2xx / 4xx / 5xx	Success / the client's fault / the server's fault.
Timeout	Giving up on a request that's taking too long.
Server-side	Anything happening where the user can't see or modify it.

Done when

You can point at a request in DevTools and narrate every part of it.

Session 8 — A real database with Supabase

~90 min

Why this matters

Everything you've built so far forgets. This is the session where data survives, and the conceptual jump from "application state" to "persistent state" is bigger than it sounds written

down. It's also where the SQL from Sessions 3 and 4 stops being an exercise and starts being how you inspect your own product.

Objectives

By the end you should be able to:

- Explain exactly where a piece of data physically lives and what path it took
- Create a table with appropriate columns and relationships
- Trace a full round trip: form → API route → database → back → display
- Run a SQL query against your own real data and get an answer you care about
- Explain what a connection string is and why it's a secret

Read

- [Supabase — Getting Started](#)
- [Supabase — Tables and Data](#)

Build

Create the tables you designed in Session 4:

```
None
companies: id, name, user_id, created_at
notes:      id, company_id, content, created_at
```

Then wire both directions:

```
None
Add Company      → POST → backend → Supabase → INSERT
Companies page →      backend → Supabase → SELECT → display
```

The test that makes it real

Add a company. Close the browser. Stop the dev server entirely. Restart everything.

It's still there.

That's persistence. Obvious in the abstract, different when you watch it.

Then use your SQL

Open the Supabase SQL editor and run the queries from Session 4 against your own data. Join companies and notes. Count notes per company. This is the moment SQL becomes a tool rather than a subject.

Go deeper

- [Supabase — Database Functions](#) — skim, just to know the capability exists.
- [Supabase — Realtime](#) — skim. Useful for understanding what "live updates" costs architecturally.
- [The Twelve-Factor App](#) — read factors III (config) and IV (backing services). Short, opinionated, and it's where a lot of modern deployment convention comes from.

Glossary

Term	What it means
ORM / query builder	A library that writes SQL for you from code.
Connection string	The credential-bearing address of your database. A secret.
Persistence	Data surviving a restart.
Seed data	Fake rows for development.
Backup / restore	What saves you when something goes badly wrong.
Transaction	A group of changes that all succeed or all fail together.
Managed service	Someone else runs the infrastructure; you use it.
Backing service	Any external resource your app depends on — database, cache, mail.

Done when

You can trace a single piece of data from the form field it was typed into all the way to the row it occupies, and back to the screen.

Session 9 — Authentication and authorization

~90 min

Why this matters

Conflating these two is the most common security mistake non-technical people make, and the consequences are severe. It's also unavoidable vocabulary — "auth" comes up constantly, and knowing which auth someone means is half of following the conversation.

The specific idea worth internalizing: **hiding something in the frontend is not security**. Anyone can open DevTools and see everything your browser received. Security has to be enforced where the user has no control.

Objectives

By the end you should be able to:

- Explain the difference between authentication and authorization, with an example of each
- Explain what happens mechanically when someone logs in
- Explain why hiding data in the UI is not security, and what actually is
- Recognize a serious security mistake when you see one — credentials in frontend code, missing access checks
- Explain roughly what a session, a cookie, and a JWT each do

Read

- [Supabase — Auth](#) — the introduction
- [Supabase — Row Level Security](#) — skim. RLS is Postgres itself enforcing which rows a user may touch, which is a genuinely good idea to understand.
- [OWASP Top Ten](#) — read the summaries only. You want recognition, not expertise.

Build

Signup, login, logout. Then make companies belong to users — and verify a second account genuinely cannot see the first account's data. Actually test it with two accounts. Assumptions here are how breaches happen.

Ask Claude

Walk me through everything that happens technically when I enter my email and password and click Login.

Where does the user's identity live on subsequent requests?

Explain cookies, sessions, JWTs, authentication, and authorization at the depth a founder needs — not more.

Go deeper

- [Auth0 — Authentication vs Authorization](#) — clean explanation of the distinction.
- [What is OAuth?](#) — the protocol behind "Sign in with Google."
- [Have I Been Pwned](#) — not a resource exactly, but browsing what gets leaked in breaches is a fast way to build the right instincts.

Glossary

Term	What it means
Authentication	Proving who you are.
Authorization	What you're permitted to do once you're in.
Session	The server's memory that you're currently logged in.
Cookie	A small piece of data the browser sends back on every request.
JWT	A signed token carrying identity claims, verifiable without a database lookup.
OAuth	The protocol behind third-party sign-in.
Hashing	One-way scrambling. Passwords are stored hashed and are never readable.
Salt	Random data added before hashing so identical passwords don't match.
Row Level Security (RLS)	The database enforcing who can see which rows.
SQL injection	Attacker-supplied text getting executed as a query.
Least privilege	Give every component the minimum access it needs.

Term	What it means
Access token / refresh token	Short-lived proof of identity / the thing that renews it.

Done when

You can explain to someone else why frontend-only access control is not security.

Session 10 — Deployment, environments, and secrets

~75 min

Why this matters

Two things here. First, understanding the deploy pipeline demystifies a large part of how software companies actually operate day to day — staging, production, rollbacks, CI. Second, and more urgent: secrets management is the one security topic where a single mistake can end a company. Leaked credentials are not a hypothetical.

Objectives

By the end you should be able to:

- Explain the full chain from your laptop to a public URL
- Explain the difference between local, staging, and production, and why all three exist
- Explain why an API key must never appear in frontend code, and what happens if it does
- Set up an environment variable and know where it's read from
- Explain what a rollback is and why it's the first move when production breaks

Read

- [Vercel — Getting Started](#)
- [Vercel — Environment Variables](#)
- [The Twelve-Factor App — Config](#) — one page. The canonical argument for why config lives outside code.

Deploy

Connect GitHub → Vercel → deploy. Then internalize the chain:

None

Your laptop → git push → GitHub → Vercel builds → public URL

Note that GitHub is the middleman. Vercel never talks to your machine.

The part that actually matters

Your database password and API keys must never appear in your source code, because your source code is on GitHub and your frontend code is downloadable by every visitor to your site.

None

Browser = user-controlled, fully inspectable, never trusted with secrets
Server = your environment, where secrets can live

If you take one security idea from this course, take this one.

Exercise

Send the URL to someone. Have them sign up, add a company, add a note. Then find the row they created in your Supabase dashboard.

Then, separately: open your deployed site, open DevTools → Sources, and look at what JavaScript the browser actually downloaded. Everything there is public. That's why the boundary matters.

Go deeper

- [Cloudflare — What is a CDN?](#) — why your site loads fast from anywhere.
- [GitHub — About secret scanning](#) — GitHub actively looks for leaked keys, which tells you how common the mistake is.

Glossary

Term	What it means
Environment variable	Configuration supplied at runtime rather than written in code.
Secret	Any credential — API key, database password, token.
.env file	Local secrets. Always gitignored.

Term	What it means
Local / staging / production	Your machine / a rehearsal copy / the real thing.
Build	Compiling and bundling before serving.
Deploy	Making a new version live.
Rollback	Reverting to the last version that worked. Usually the right first move.
CI/CD	Automation that tests and deploys on every push.
CDN	Distributed servers caching content near users.
DNS records	What point your domain at your host.
Uptime / SLA	How often it works / what you've contractually promised.
Zero-downtime deploy	Shipping without the site going dark.

Done when

You can explain, to a non-technical person, why `OPENAI_API_KEY=...` in frontend code is an emergency.

PART III — AI-NATIVE SOFTWARE

Sessions 11–14. Go meaningfully deeper here than a typical non-technical founder.

Session 11 — How AI models actually work

~75 min

Why this matters

Every startup conversation in 2026 touches this, and the vocabulary is new enough that real fluency is genuinely differentiating — most people using these words are repeating them. Understanding tokens and context windows in particular explains a huge amount of downstream product behavior: why some features are expensive, why responses get slower with longer inputs, why "just feed it all our documents" doesn't work, why models forget things mid-conversation.

This session is conceptual. No building. Get the mental model before you wire anything up.

Objectives

By the end you should be able to:

- Explain what a token is and estimate roughly how many are in a document
- Explain what a context window is and what happens when you exceed it
- Explain, in one sentence each, the difference between training, fine-tuning, and inference
- Reason about why one AI feature costs 50× another
- Explain what a hallucination is and why the model can't tell it's happening
- Push back on "we'll just use AI for that" with a specific question

Read

- [Anthropic — Introduction to Claude](#) — orientation
- [Anthropic — Context windows](#) — the core mechanic
- [OpenAI — Tokenizer](#) — paste text in, watch it split into tokens. Five minutes here beats an hour of reading about tokens.

The concepts

Tokens. Models don't read words; they read tokens — chunks of roughly $\frac{3}{4}$ of a word in English. Everything is priced and limited in tokens: what you send, what comes back. A 10-page document is roughly 5,000 tokens. This is the unit of both cost and capacity.

Context window. How much the model can hold in mind at once, measured in tokens. Everything the model "knows" about your specific situation has to fit — the system prompt, conversation history, retrieved documents, the question. Exceed it and something gets dropped.

Training vs inference. Training is the enormously expensive one-time process that produced the model. Inference is running it to get an answer, which is what you pay for per call.

Fine-tuning is additional training on your own data — usually the wrong solution, and worth knowing that so you can question it when someone proposes it.

Cost and latency. Input tokens are cheaper than output tokens. Bigger models cost more and respond slower. A product that sends 50,000 tokens per request behaves very differently, economically, from one that sends 500. When someone pitches an AI product, "how many tokens per user action?" is a sharp question.

Hallucination. The model produces fluent, confident, wrong output. It has no internal signal distinguishing this from correct output — which is why "the AI will just check its work" isn't a plan, and why evals exist.

Non-determinism. The same input can produce different output. This breaks most conventional software assumptions and is the reason testing AI features is a genuinely hard problem.

Do

Open the tokenizer. Paste in a paragraph of your own writing, then a table of numbers, then a code snippet. Notice how differently they tokenize.

Then take a product you know and estimate: how many tokens per user action? What would that cost at 10,000 users a day? Look up current pricing and actually do the arithmetic. This is the calculation nobody does and everyone should.

Go deeper

- [Anthropic — Pricing](#) — look at the per-token numbers so the economics are concrete
- [Simon Willison's blog](#) — the best ongoing practical writing on what these models actually do
- [Andrej Karpathy — Intro to Large Language Models](#) — one hour, and the clearest available explanation of what's happening inside. Optional but excellent.

Glossary

Term	What it means
Token	The unit models read and write. Roughly $\frac{3}{4}$ of a word.
Context window	How many tokens the model can consider at once.
Prompt	What you send the model.

Term	What it means
System prompt	Standing instructions framing every request in a conversation.
Completion / output	What comes back.
Inference	Running the model to get an answer. What you pay for.
Training	The one-time process that created the model.
Fine-tuning	Further training on your own data. Usually unnecessary.
Temperature	How much randomness in the output. Lower is more predictable.
Latency	Response time. Models are slow relative to databases.
Streaming	Sending output word by word so it feels faster.
Hallucination	Fluent, confident, wrong.
Non-deterministic	Same input, potentially different output.
Multimodal	Handles more than text — images, audio, documents.
Model family / version	Different sizes and generations, with different cost/quality tradeoffs.

Done when

Someone pitches an AI product and you can immediately ask two sharp questions about cost and reliability.

Session 12 — Putting an LLM inside your product

~90 min

Why this matters

The shift here is from *using* AI to *building with* it. When a model becomes a component in your architecture rather than a tool you chat with, a set of new questions appear: where does the key live, what happens when the call fails, how long does the user wait, what if the output is garbage. Every AI product wrestles with these.

Objectives

By the end you should be able to:

- Explain the difference between using ChatGPT and building software that calls a model
- Trace a full request through your app: click → backend → model → database → screen
- Explain why the browser cannot call the model API directly
- Handle a failed or slow model call without the app breaking
- Explain what a prompt actually is at the API level — just text you're constructing in code

Read

- [Anthropic — Get started with the API](#) — you already have a Claude account
- or [OpenAI — Developer Quickstart](#)

Either is fine. The concepts are identical and transfer completely.

Build

An "Analyze Company" button:

```
None
Click
↓
POST /api/analyze
↓
backend fetches that company's notes from Postgres
↓
sends them to the model with a prompt
↓
returns the response
↓
display it
```

Notice what the backend is doing: assembling context from your database and constructing a prompt in code. That's most of what "AI engineering" is in practice.

The conceptual question to sit with

Why can't the browser call the model API directly with your key?

Because anything in the browser is visible to the user. An exposed API key is someone else's free compute on your credit card, and it will be found within hours. The backend exists partly *because* it's the only place a secret can live.

Also handle

- What does the user see while waiting? Model calls take seconds, not milliseconds.
- What happens if the call fails or times out?
- What happens if the model returns something unusable?

These three questions are most of the difference between a demo and a product.

Go deeper

- [Anthropic — Prompt engineering overview](#) — practical and short
- [Anthropic — Streaming](#) — why good AI products feel fast despite being slow

Glossary

Term	What it means
API key	Your credential for the model provider. A secret, server-side only.
Prompt construction	Assembling the text you send, usually from database content plus instructions.
Context assembly	Deciding what information to include in a request.
System vs user message	Standing instructions vs the specific request.
Rate limit	How many requests the provider allows per minute.
Retry / backoff	Trying again after a failure, with increasing delays.
Timeout	Giving up on a slow call.
Loading state	What the user sees while waiting.

Term	What it means
Graceful degradation	The app still works, less well, when a component fails.

Done when

You can draw the full path of an AI feature through your system and name what could fail at each step.

Session 13 — Structured output, tools, and agents

~90 min

Why this matters

Your main "AI startup architecture" session. This is where the marketing vocabulary — agents, tool use, orchestration — becomes concrete mechanism. It's also where most of the actual differentiation lives in AI products right now, so understanding it well is directly job-relevant.

Objectives

By the end you should be able to:

- Explain why structured output matters more than better prose
- Explain the tool-calling loop mechanically, without hand-waving
- Explain the difference between an agent and a workflow, and argue for when each is appropriate
- Say what an eval is and why AI products can't be tested conventionally
- Hear "we're building an agent" and ask the right follow-up question

Part 1 — Structured output

Instead of a paragraph of prose, constrain the model to return exactly this:

```
JSON
{
  "summary": "...",
  "strengths": ["...", "..."],
  "risks": ["...", "..."],
  "score": 8
}
```


Then store it in your database.

Why this is the key idea: prose can't be stored usefully, sorted, filtered, aggregated, or displayed in a table. Structured data can. This is most of what separates an AI *feature* from an AI *demo* — the demo produces impressive text, the feature produces data your product can actually use.

Read: [Anthropic — Tool use](#) or [OpenAI — Structured Outputs](#)

Part 2 — Tool calling

```
None
User asks a question
↓
Model determines it needs data it doesn't have
↓
Model emits a tool call: get_company_notes(id)
↓
YOUR code runs the actual query
↓
Result goes back to the model
↓
Model answers using it
```

The model never touches your database. It requests; your code decides whether and how to comply. That distinction is the whole security model of agentic systems, and it's worth being crisp about.

Implement exactly **one** tool. Don't build a multi-agent system. Watch the sequence, log every step, understand it.

Part 3 — Evals

Conventional software: same input, same output, so you write tests. AI: same input, different output, so tests don't work. Evals are the replacement — a set of cases with graded outputs, run repeatedly, tracked over time.

Every serious AI company has an eval system and most of them consider it their real moat. Knowing this puts you ahead of most people using the word "agent."

Go deeper

- [Anthropic — Building Effective Agents](#) — **read this one properly, not as a skim.** The single best piece written on when an agent is the right architecture and when plain deterministic code is better. It will save you from a category of expensive mistake.
- [Anthropic — Define your success criteria](#) — how to think about evaluating model output
- [Hamel Husain — Your AI Product Needs Evals](#) — the practical case, from someone who does this for a living

Glossary

Term	What it means
Structured output	Constraining the model to a defined JSON shape.
Schema	The definition of that shape.
Tool / function calling	The model requesting that your code do something.
Agent	A model in a loop, using tools, choosing its own next step.
Workflow	A predetermined sequence with model calls in it. Usually better than an agent.
Orchestration	Coordinating multiple model and tool calls.
Eval	A test suite for non-deterministic output.
Guardrail	A check on what goes in or what comes out.
Human in the loop	A person approving before an action executes.
Deterministic vs probabilistic	Same input, same output — versus not.
Chain of thought	Having the model reason step by step before answering.
Prompt injection	Untrusted input containing instructions the model follows. A real attack.

Done when

"Agent," "tool call," "structured output," and "eval" each name a specific mechanism in your head, not a vibe.

Session 14 — Retrieval, embeddings, and AI system architecture

~75 min

Why this matters

RAG is the architecture behind most enterprise AI products, and it's also the answer to the most common question you'll hear: "can we just point AI at all our documents?" Understanding why the answer is "not directly" — and what the actual approach is — is genuinely useful vocabulary.

Conceptual only. Don't implement a vector database.

Objectives

By the end you should be able to:

- Explain why you can't just paste 100,000 documents into a prompt
- Explain what an embedding is without using the word "vector" as an explanation
- Describe the RAG pipeline end to end
- Explain why RAG often works worse than expected, and name two reasons
- Explain why "AI product" and "LLM" are not the same thing

The problem

You have 100,000 documents and a context window that fits maybe 200 of them. So:

```
None
User question
  ↓
Convert the question to an embedding
  ↓
Find the most similar document chunks
  ↓
Put only those in the context
  ↓
Model answers using them
```

That's RAG. Less exotic than it sounds: it's search, followed by a prompt. Most of the difficulty is in the search half, not the model half — which is why RAG systems usually underperform initially.

Read

- [Anthropic — Embeddings](#) or [OpenAI — Embeddings](#)
- [Anthropic — Contextual Retrieval](#) — why naive RAG underperforms and what actually helps. This is the useful one.

The framing that matters

```
None
AI product = conventional software
               + model calls
               + retrieval
               + tools
               + data
               + evaluation
               + interface
```

Not: `AI product = LLM`.

The most common failure among non-technical founders is assuming the model is the product. In a real AI application the model calls are usually a small fraction of the code and a smaller fraction of the difficulty.

Go deeper

- [Pinecone — Vector Database explainer](#) — clear, vendor-written but honest
- [Chip Huyen's blog](#) — the best writing on AI systems as engineering rather than research

Glossary

Term	What it means
Embedding	Text converted into a list of numbers that captures meaning.
Vector	That list of numbers.
Semantic similarity	Closeness in meaning, measured as closeness of vectors.

Term	What it means
Vector database	Storage optimized for finding similar vectors fast.
Retrieval	Finding the relevant subset of your data.
RAG	Retrieval-Augmented Generation. Search, then prompt.
Chunking	Splitting documents into pieces small enough to retrieve usefully.
Reranking	A second pass reordering retrieved results by relevance.
Hybrid search	Combining keyword search with semantic search. Usually beats either alone.
Context stuffing	Cramming everything in and hoping. Expensive, degrades quality.
Grounding	Tying model output to source documents you can cite.

Done when

You can explain to a non-technical person why a company with a lot of internal documents can't just paste them into ChatGPT, and what they'd do instead.

PART IV — SYSTEMS AND JUDGMENT

Sessions 15–17. The part that makes you useful rather than merely fluent.

Session 15 — Debugging and observability

~90 min

Why this matters

The most important session in the course for your specific goal. AI writes code very fast; the scarce skill now is figuring out why the system doesn't behave as intended. This is also what separates someone who can supervise engineering work from someone who can only generate it.

Practically: you will hit a bug that AI can't fix for you, because fixing it requires knowing which layer is broken, and only you can see the whole system.

Objectives

By the end you should be able to:

- Given a broken feature, name which layer failed and cite the evidence — before writing any code
- Read a stack trace and identify where the error originated
- Find and read server logs
- Reproduce a bug reliably, which is usually half the work
- Resist the urge to guess, and check instead

The rule for this session

When something breaks, don't ask Claude Code to fix it. Ask:

Don't change any code yet. Help me identify which layer is failing, what evidence we have, and the three most likely causes.

Then investigate yourself. The fix is worth less than the diagnosis.

Read

- [Chrome DevTools — Overview](#)
- [Chrome DevTools — Network](#)
- [Chrome DevTools — Console](#)

Break your app on purpose

Four failures, one at a time. For each, find the evidence *before* fixing:

1. **Wrong API URL** — find the failed request in the Network panel. What status? What does the response say?
2. **Misspelled API key** — find the backend error. Note where it surfaces and where it doesn't.

3. **Query a column that doesn't exist** — read the database error message properly. Database errors are unusually informative.
4. **Return malformed JSON** — watch what the frontend does. Note that the failure appears far from its cause.

That fourth one is the important lesson: **where an error appears is often not where it originated.**

The tree to use every time

```
None
Something's wrong
↓
Did the UI even try?           → browser console
↓
Did a request go out?         → Network panel
↓
What status came back?       → Network panel
↓
Did the backend receive it?   → server logs
↓
Did the DB or API call work?  → server logs, DB dashboard
↓
What exactly came back?      → response body
```

Almost every bug localizes by walking this in order. The skill is refusing to skip steps.

Go deeper

- [Sentry docs](#) — skim. Understand what error monitoring is and why every company has it.
- [PostHog docs](#) — skim. Same, for product analytics. Knowing what these tools show you means knowing what questions you can ask at a company.
- [Julia Evans' debugging zines](#) — illustrated, genuinely excellent, and the best available writing on debugging as a discipline.

Glossary

Term	What it means
Stack trace	The chain of function calls that led to an error.
Console	Where browser-side errors and logs appear.
Server logs	Where backend errors appear.

Term	What it means
Observability	Being able to tell what a running system is doing.
Monitoring / alerting	Watching for problems / being told about them.
Error monitoring	Automatic capture of production errors.
Reproduce	Making a bug happen on demand.
Regression	Something that used to work and no longer does.
Root cause	The actual reason, as opposed to the symptom.
Edge case	The input nobody planned for.
Race condition	A bug depending on timing. Miserable to reproduce.
Silent failure	Something broke and nothing said so. The worst kind.

Done when

You can take a broken feature and produce a diagnosis with evidence, without touching any code.

Session 16 — Reading a codebase you didn't write

~90 min

Why this matters

Badly underrated, and directly job-relevant. It's what lets you inspect reality rather than relying on someone's verbal summary of it. It's also the difference between "the engineer said it'd be hard" and "I looked, and I understand why it'd be hard."

At a startup, this skill compounds fast. Within a month of reading your own company's code you'll follow every conversation.

Objectives

By the end you should be able to:

- Open an unfamiliar repo and find where a specific feature lives, in about ten minutes
- Identify what external services a project depends on, without asking
- Trace one user action from the interface to the database and back
- Form a rough opinion about a codebase's quality and organization
- Know which files to read first

Do

Pick a real open-source Next.js project on GitHub — something with a few thousand stars, not a tutorial repo. Then answer these **without asking anyone**:

1. Where does the app start? What's the entry point?
2. Where does the UI live?
3. Where are the API routes or server functions?
4. Where is the database schema defined?
5. What external services does it use?
6. Where do credentials get configured?
7. Trace one feature end to end — from the button to the database and back.

You will struggle. That's the exercise. Give it a full 45 minutes before asking for help.

Then

Ask Claude Code the same questions. Compare its answers to yours. Where you were wrong, work out what signal you missed — that's the actual learning.

The files that tell you the most, fastest

- **README.md** — what the maintainers thought mattered
- **package.json** — every dependency, i.e. every external thing this touches
- **.env.example** — every service it connects to
- **folder structure** — usually the architecture in miniature
- **recent commits** — what's actively being worked on

Go deeper

- Read the codebase of something you use. [Cal.com](#) and [Dub](#) are both real Next.js products, open source, and well-organized enough to learn from.
- Do this exercise a second time in a month. The gap between attempts is a good measure of progress.

Glossary

Term	What it means
Entry point	Where execution begins.
Dependency	External code the project relies on.
Monorepo	Multiple projects in one repository.
Boilerplate	Necessary, uninteresting setup code.
Abstraction layer	Code whose job is hiding other code's complexity.
Config	Settings separated from logic.
Convention over configuration	The framework assumes defaults so you write less setup.
Tech debt	Shortcuts taken earlier that cost you now.
Refactor	Restructure without changing behavior.
Legacy code	Code nobody wants to touch. Every company has some.
Coupling	How much one part depends on another. Less is better.

Done when

You can open an unfamiliar repo and locate a specific feature within about ten minutes.

Session 17 — System design, scoping, and judgment

~90 min

Why this matters

Now the vocabulary will land, because you've hit the problems it describes. Reading this material on day one would have given you words with nothing underneath — "we'd probably want a queue there" sounds fine until someone asks why.

The scoping half of this session isn't on any standard curriculum, and it's the skill that determines whether "I'll hire someone technical" is a strategy or a way to get taken advantage of. You cannot hire well for something you can't evaluate.

Objectives

By the end you should be able to:

- Draw the architecture of a product you use, from its behavior
- Explain what a cache, a queue, and a worker each solve
- Identify the likely bottleneck in a system as it scales
- Estimate how long a feature takes, within an order of magnitude
- Hear "that'll take three months" and know what question to ask next
- Follow a system design conversation between engineers without translating

Concepts

Term	What it means
Server	A process handling requests.
Database	Persistent structured data.
Object storage	Files — images, video, PDFs. Different from a database, and cheaper.
Cache	Fast temporary storage so expensive work isn't repeated.
Queue	Work waiting to be done later.
Worker	A process doing queued work in the background.
Load balancer	Distributes traffic across multiple servers.
CDN	Content served from near the user.
Latency	How long one thing takes.
Throughput	How much work per unit time.
Horizontal / vertical scaling	More machines / bigger machines.
Monolith / microservices	One deployable unit / many.

Term	What it means
Synchronous / asynchronous	The caller waits / it doesn't.
Bottleneck	The one component limiting everything else.
Single point of failure	The thing whose death kills the system.
Idempotency	Safe to run twice. Matters enormously with queues and retries.
Eventual consistency	Different parts of the system briefly disagree, then converge.

Read

- [Cloudflare — What is a CDN?](#)
- [Cloudflare — What is load balancing?](#)
- [Cloudflare — What is caching?](#)
- [AWS Well-Architected Framework](#) — skim the five pillars only. The takeaway is that architecture is tradeoffs among reliability, security, performance, cost, and operations. Don't read the whole thing.

Exercise A — design four systems, 10 minutes each

- 1. Your own CRM.** Draw it completely, from memory, before looking at anything.
- 2. Dropbox.** Where do files live? Where does metadata live? What happens when a second device wants the file? Why are those two storage systems different?
- 3. Uber.** Where does location data go? How does the system find nearby drivers? How do rider and driver stay in sync? What breaks at scale?
- 4. An AI customer support agent.** Where does customer data come from? What tools can the model call? What happens when it's uncertain? What requires human approval? What do you log?

Correctness isn't the point. Decomposing a product into technical components is.

Exercise B — scoping and estimation

For each, estimate how long a competent engineer would take. Write your number down *first*, then check your reasoning with Claude and see how far off you were.

- Add a "delete company" button

- Add CSV export of all notes
- Add a second user role with different permissions
- Change the primary key type on an existing table with data in it
- Switch from Supabase to a different database
- Add real-time updates so two users see each other's changes live
- Add a full audit log of every change
- Add support for a second language

You'll be badly wrong on at least two. **The gap between your estimate and reality is the lesson** — that's what calibrates you for the moment an engineer says "three months" and you need to know whether that's honest, padded, or optimistic.

Pay attention to which ones surprised you. Usually it's the ones that sound simple (changing a primary key, adding a language) and are structurally invasive.

Go deeper

- [System Design Primer](#) — comprehensive and free. Read the top-level sections; it's aimed at interviews so skip the deep dives.
- [Designing Data-Intensive Applications](#) — the standard text. Read it in about three months, once you've hit enough real problems for it to land.

The final test

Draw your CRM architecture from memory, then answer:

1. What happens when a user loads the page?
2. What happens when they submit a company?
3. Where is that company stored, physically?
4. What SQL runs, roughly?
5. What is JSON doing in this system?
6. What does the backend do that the frontend can't?
7. Why is authentication necessary, and what would break without it?
8. What happens when the Analyze button is clicked, step by step?
9. Where does the API key live and why there specifically?
10. What does Vercel do? Supabase? GitHub? Why three separate things?
11. What does a 500 mean, and how would you find the cause?
12. What breaks first at 100,000 users?
13. What would a cache help with here? What wouldn't it help with?
14. What would a queue help with here?
15. What would "adding an agent" actually mean in this system?
16. How would you integrate a third-party service like Salesforce?
17. How many tokens does one Analyze click cost, roughly, and what does that mean at scale?

18. If the AI analysis started returning worse results next month, how would you know?

Discuss 14 of 18 intelligently and you've hit the target.

What NOT to study

Protecting your time is part of the curriculum. Skip entirely:

LeetCode · sorting algorithms · linked lists · binary trees · dynamic programming · C/C++ · operating system internals · compiler theory · Kubernetes · advanced Docker · advanced AWS · advanced React patterns · advanced CSS · implementing microservices · distributed consensus · database internals · training neural networks · linear algebra for ML · transformer mathematics

None of these are unimportant. All of them are low marginal value for what you're trying to become. You learn any of them when a specific project forces you to — never preemptively.

After the course

Immediately: rebuild something in your own domain. Interconnection queue data, public power bond issuance, data center siting. Same stack, real data, one real user. That project is where your background stops being a résumé line and becomes something nobody else can replicate.

Within a month: read code at whatever company you land at. Nothing accelerates fluency like a real codebase with real stakes and real people to ask.

Around three months: [Designing Data-Intensive Applications](#). The standard system design text, and readable only once you've accumulated enough real problems for it to attach to.

Ongoing: keep the glossary. Add to it every time a term comes up in a meeting that you nodded through. That single document, more than any project, is what turns into fluency.

The four questions

Whenever Claude Code generates something you don't fully follow, ask these instead of trying to memorize the code:

1. What problem is this solving?
2. Where does it sit in the larger system?
3. What goes in and what comes out?
4. What happens if it fails?

The goal at the end of this is not *"I know how to code."*

It's: *"I understand how software products are assembled, I can hold my own in a conversation with engineers, and with AI assistance I can build, inspect, modify, deploy, and debug a simple one myself."*